Vision Language Models

AI623 - Programming Assignment 3

This assignment is not graded and has no deliverables. There is no submission for this assignment, and no marks are awarded for producing code, plots, or results. Instead, the assessment of the PA content will occur through two separate components:

- **a viva** (oral examination), and
- **a written test** (a short quiz).

The purpose of this assignment is therefore *preparatory*: it is a structured way to build the understanding and implementation intuition that you will be asked to demonstrate in the viva and written test. **Please aim to complete this assignment by 30th April 2026.**

Maintain a working document (strongly encouraged). Even though nothing is submitted, you are strongly encouraged to maintain a "working document" (notes + screenshots + plots + short explanations) as you progress.

Structure of this assignment. The assignment covers two complementary approaches to connecting visual and language representations:

- **Part A** (Continuous connector VLMs): A frozen CLIP vision encoder feeds patch tokens through a learned connector into a pretrained LM. Dataset: CIFAR-10 subset.
- **Part B** (Discrete VQ-VAE): A VQ-VAE compresses synthetic images into discrete codebook indices appended directly to the LM's vocabulary. The LM handles vision and language in a single unified token stream. Dataset: procedurally generated synthetic images.

Required reading. Work through Section 2.3 before attempting any task.

Contents

1 Allowed Tools and Constraints **3**

2 Background and Notation **4**
2.1 Notation Dictionary . . . 4
2.2 The Two Architectures at a Glance . . . 4
2.3 Resources . . . 5

3 Analytical Problems **6**
3.1 Problem 0: CLIP and SigLIP . . . 6
3.2 Problem 1: VLM Objectives and Connector Design . . . 9
3.3 Problem 2: VQ-VAE Theory . . . 10
3.4 Problem 3: Unified Vocabulary and Token Sequences (Part B) . . . 11
3.5 Problem 4: Catastrophic Forgetting and Mixed Objectives . . . 12
3.6 Problem 5: Modality Gap and Cross-Architecture Comparison . . . 12

Coding: Part A — Continuous Connector VLMs **14**

4 Coding Tasks — Part A **14**
4.1 Dataset: CIFAR-10 Subset . . . 14
4.2 Task A-C0: Data Pipeline and Model Loading . . . 14
4.3 Task A-C1: Phase 1 — Connector Initialisation . . . 15
4.4 Task A-C2: Phase 2 — SFT with Language Replay . . . 16
4.5 Task A-C3: Phase 3 — VQA Alignment . . . 16
4.6 Task A-C4: Evaluation . . . 16
4.7 Task A-C5: Modality Gap Analysis . . . 17
4.8 Task A-C6: Ablations (choose ≥ 1) . . . 17

Coding: Part B — Discrete VQ-VAE **18**

5 Coding Tasks — Part B **18**
5.1 Dataset: Synthetic Images . . . 18
5.2 Task B-C0: Dataset Generation and Model Loading . . . 19
5.3 Task B-C1: VQ-VAE Training and Codebook Analysis . . . 19
5.4 Task B-C2: Vocabulary Expansion . . . 21
5.5 Task B-C3: Tokenisation Pipeline . . . 21
5.6 Task B-C4: Mixed-Objective Fine-Tuning . . . 22
5.7 Task B-C5: Evaluation . . . 23
5.8 Task B-C6: Focused Ablations (choose ≥ 1) . . . 24

Appendix **25**

1 Allowed Tools and Constraints

- **Permitted libraries**: PyTorch, HuggingFace `transformers`, `peft` (LoRA), `accelerate`, `datasets` (Alpaca), `torchvision` (CIFAR-10), and standard scientific Python (`numpy`, `matplotlib`, `scikit-learn`).
- **Not permitted**: any library providing a ready-made VLM trainer, VQ-VAE, or multimodal training loop.
- **Implement from scratch** (Part A): the MLP connector architecture; all training phases; the mixed objective with Alpaca replay; all metrics. (Q-Former and gated cross-attention are covered analytically in Problem 1.2.)
- **Implement from scratch** (Part B): the VQ-VAE (encoder, vector quantiser with gradient and EMA modes plus dead-code restart, decoder); vocabulary-expansion and embedding-overlay logic; mixed-objective training loop; all evaluation metrics.
- **Language model (both parts)**: `HuggingFaceTB/SmolLM2-360M-Instruct` (360 M params, $d_{\mathrm{LM}} = 960$, 49 152 vocabulary tokens, 24 layers). Load in `float16`; use mixed-precision training (`GradScaler`).
- **Vision encoder** (Part A only): `openai/clip-vit-base-patch32`, frozen. Extract `last_hidden_state` patch tokens $Z \in \mathbb{R}^{49\times 768}$ (discard CLS at index 0).
- **LoRA configuration (both parts)**: $r = 16$, $\alpha = 32$, targets `q_proj, k_proj, v_proj, o_proj`, dropout 0.05.
- **Text-replay dataset (both parts)**: 1 000 examples from `tatsu-lab/alpaca`. Mandatory for all mixed-training runs. Compute baseline perplexity PPL_0 before any fine-tuning.

2 Background and Notation

2.1 Notation Dictionary

Symbol	Meaning
Shared	
$d_{\mathrm{LM}} = 960$	SmolLM2-360M hidden dimension
$V_{\mathrm{txt}} = 49\,152$	Original LM vocabulary size
r	LoRA rank (default 16)
λ	Language-replay loss weight
R	Forgetting ratio: $\mathrm{PPL}^{\mathrm{fine}}/\mathrm{PPL}_0$
Part A — Continuous connector	
$x^v \in \mathbb{R}^{H\times W\times 3}$	Input image (CIFAR-10, 32×32)
$Z \in \mathbb{R}^{49\times 768}$	CLIP patch tokens ($d_v = 768$, CLS discarded)
$V \in \mathbb{R}^{M\times d_{\mathrm{LM}}}$	Visual tokens after connector ($M \leq 49$)
Θ_C	Connector parameters
r_{norm}	Norm ratio: $\mathbb{E}\lVert V_i\rVert_2/\mathbb{E}\lVert T_j\rVert_2$
MG	Modality gap on unit sphere $\in [0, 2]$
Part B — Discrete VQ-VAE	
$x \in \mathbb{R}^{16\times 16\times 3}$	Synthetic input image
$z_e = E_\phi(x) \in \mathbb{R}^{4\times 4\times 64}$	Pre-quantisation encoder output
$\mathcal{C} = \{e_k\}_{k=1}^{K},\ e_k \in \mathbb{R}^{64}$	VQ-VAE codebook ($K = 256$)
$z_q = Q(z_e)$	Quantised latent (nearest-neighbour lookup)
$\hat{x} = D_\psi(z_q)$	Decoded reconstruction
β	Commitment-loss weight
γ_{img}	Image-generation loss weight

2.2 The Two Architectures at a Glance

	Part A (Continuous)	**Part B (Discrete)**
Vision encoder	CLIP ViT-B/32 (pretrained, frozen)	VQ-VAE encoder (trained from scratch)
Visual repr.	Continuous embedding $V \in \mathbb{R}^{M\times 960}$	Discrete token IDs $\in \{1, \ldots, K\}$
LM interface	`inputs_embeds` concatenation	Vocabulary expansion + `input_ids`
Reconstruction	Not possible	Yes, via VQ-VAE decoder

2.3 Resources

Core Papers

1. **LLaVA** — Liu et al., "Visual Instruction Tuning", NeurIPS 2023.
2. **BLIP-2** — Li et al., "BLIP-2", ICML 2023.
3. **Flamingo** — Alayrac et al., "Flamingo", NeurIPS 2022.
4. **VQ-VAE** — van den Oord et al., "Neural Discrete Representation Learning", NeurIPS 2017.
5. **VQ-GAN** — Esser et al., "Taming Transformers", CVPR 2021.
6. **Chameleon** — Team Chameleon, arXiv 2024.
7. **LoRA** — Hu et al., ICLR 2022.
8. **Modality gap** — Liang et al., NeurIPS 2022.
9. **CLIP** — Radford et al., ICML 2021.

Background Reading

1. HuggingFace, "Vision Language Models Explained"
2. CLIP model documentation
3. HuggingFace PEFT: LoRA configuration
4. SmolLM2-360M-Instruct model card

3 Analytical Problems

Complete all problems below **before** starting any coding task. Problem 0 covers CLIP and SigLIP foundations; Problems 1–2 address Part A (continuous connectors) and Part B (VQ-VAE) respectively; Problem 3 covers vocabulary expansion and positional encoding (Part B); Problems 4–5 span both parts (forgetting and modality gap).

3.1 Problem 0: CLIP and SigLIP

CLIP and SigLIP are *contrastive* vision-language models. They do not reconstruct images, and, as standalone vision-language encoders, they do not represent images as discrete token sequences inside an autoregressive language model. Instead, they learn two encoders, an image encoder and a text encoder, whose outputs are mapped into a *shared* embedding space. Matching image-text pairs should have high similarity, while non-matching pairs should have low similarity. This makes them fundamentally different from the VQ-VAE+LM pipeline in this assignment: CLIP/SigLIP are optimised primarily for *alignment* and retrieval-style discrimination, whereas the toy multimodal LM here is built around *discrete visual tokens* and autoregressive generation.

Let an image encoder produce $u_i \in \mathbb{R}^m$ and a text encoder produce $v_i \in \mathbb{R}^m$ for the ith matched image-caption pair in a batch of size B. Assume both are ℓ_2-normalised:

$$\|u_i\|_2 = \|v_i\|_2 = 1,$$

so that similarity is cosine similarity:

$$s_{ij} = u_i^\top v_j \in [-1, 1].$$

CLIP uses a *softmax-normalised* contrastive objective over the full batch, while SigLIP replaces this with a *pairwise sigmoid-style* objective. The difference appears small at first glance, but it changes the role of negatives, the effect of batch size, and the geometry of the learned embedding space.

Problem C.1 (CLIP objective and the role of in-batch negatives)

CLIP uses a symmetric image-to-text and text-to-image contrastive loss. With temperature parameter $\tau > 0$, define logits

$$\ell_{ij} = \frac{u_i^\top v_j}{\tau}.$$

The CLIP loss for one batch is:

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\left[\frac{1}{B}\sum_{i=1}^{B}\left(-\log\frac{\exp(\ell_{ii})}{\sum_{j=1}^{B}\exp(\ell_{ij})}\right) + \frac{1}{B}\sum_{i=1}^{B}\left(-\log\frac{\exp(\ell_{ii})}{\sum_{j=1}^{B}\exp(\ell_{ji})}\right)\right]. \tag{C1}$$

(a) Consider only the image-to-text term for a fixed image u_i:

$$\mathcal{L}^{(i)}_{\text{I}\to\text{T}} = -\log\frac{\exp(\ell_{ii})}{\sum_{j=1}^{B}\exp(\ell_{ij})}.$$

Show that minimising this loss encourages ℓ_{ii} to increase relative to ℓ_{ij} for all $j \neq i$. Explain carefully why the other captions in the same batch behave as *implicit negatives* even though no explicit negative labels were provided.

(b) Derive the softmax probability

$$p_{ij} = \frac{\exp(\ell_{ij})}{\sum_{k=1}^{B} \exp(\ell_{ik})}$$

and show that

$$\frac{\partial \mathcal{L}_{\mathrm{I}\to\mathrm{T}}^{(i)}}{\partial \ell_{ij}} = p_{ij} - \mathbf{1}\{j = i\}.$$

Interpret this gradient for the positive pair (i, i) and for a negative pair (i, j), $j \neq i$.

(c) The temperature τ scales all logits. Explain analytically what happens as $\tau \to 0$ and as $\tau \to \infty$:

- how does the softmax distribution over captions change,
- what happens to gradient concentration,
- and why can too-small τ make optimisation overly sharp or unstable?

In your answer, be explicit about the fact that $\ell_{ij} = (u_i^\top v_j)/\tau$ rescales the underlying similarities.

(d) CLIP performance often improves with larger batch sizes. Give a precise explanation in terms of (C1): why does increasing B generally create a harder discrimination problem, and why can that lead to more semantically structured embeddings?

(e) **Analyse.** Suppose two captions in the same batch both correctly describe the same image (for example, paraphrases). Under the CLIP loss, one of them is treated as the positive and the other as a negative. Explain why this creates *false negatives*. What kind of semantic distortion can this induce in the shared embedding space?

Problem C.2 (A simplified SigLIP-style objective and how it differs from CLIP)

SigLIP replaces the batch-softmax objective with a pairwise sigmoid-style objective. In a simplified form, let $y_{ij} = 1$ if (i, j) is a matched pair and $y_{ij} = -1$ otherwise. For logits $\ell_{ij} = u_i^\top v_j/\tau$, consider the loss

$$\mathcal{L}_{\mathrm{SigLIP}} = \frac{1}{B^2} \sum_{i=1}^{B} \sum_{j=1}^{B} \log(1 + \exp(-y_{ij}\ell_{ij}))\,. \tag{C2}$$

This captures the key distinction from CLIP: each pair contributes its own sigmoid-based term rather than competing through one shared softmax denominator. Equivalently, positive pairs are pushed toward large logits and negative pairs toward small logits through a sigmoid/BCE-style objective.

(a) Show that for a positive pair (i, i), the loss contribution is

$$\log(1 + \exp(-\ell_{ii})),$$

while for a negative pair (i, j), $j \neq i$, it is

$$\log(1 + \exp(\ell_{ij})).$$

Explain why this means each pair contributes its own loss term, unlike CLIP where all captions for a given image compete through one softmax denominator. Be precise about the sense in which this is "independent."

(b) Compute the derivative of one pairwise term

$$\log(1 + \exp(-y\ell))$$

with respect to ℓ, and interpret its sign for $y = 1$ and $y = -1$. Why does this objective still encourage alignment of matched pairs and separation of mismatched pairs?

(c) Compare CLIP and the simplified SigLIP-style objective along the following axes:

i. dependence on batch size,

ii. sensitivity to false negatives,

iii. whether similarities for one image must sum to a probability distribution over the batch,

iv. ease of handling more than one correct caption per image.

Your answer should be analytical, not merely descriptive.

(d) In CLIP, increasing the score of one negative pair necessarily changes the relative probabilities of all pairs through the softmax normalisation. In the sigmoid-style objective (C2), this coupling is weaker. Explain why this can make training more stable when batch composition is noisy or semantically ambiguous.

(e) **Analyse.** For very large-scale training, one may want the model to learn from many weakly related image-text pairs scraped from the web. Argue which of CLIP or a sigmoid-style objective such as (C2) is better suited to such data. Your answer should explicitly discuss label noise, multiple plausible positives, and the distinction between *relative ranking* and *absolute pair scoring*.

Problem C.3 (Geometry of the shared space and implications for this assignment)

Because u_i and v_j are normalised, both CLIP and sigmoid-style vision-language objectives operate on the unit sphere in $\mathbb{R}^m$. They do not ask the model to preserve pixel-level information. Instead, they ask the model to preserve *semantic relations* relevant to matching language with images.

(a) Explain why a contrastive objective encourages *semantic invariance*. For example, two images of the same object under different lighting or background conditions may be pushed close to the same caption. Why is this desirable for retrieval but potentially harmful for faithful image reconstruction?

(b) Compare the information preserved by:

i. a VQ-VAE code index map $\mathbf{i} \in \{1, \ldots, K\}^{4\times 4}$,

ii. a CLIP/SigLIP embedding vector $u \in \mathbb{R}^m$.

Which one is more likely to preserve spatial layout? Which one is more likely to discard low-level texture? Justify your answer from the training objective, not from intuition alone.

(c) Suppose you feed a CLIP image embedding into a language model by projecting it once into the LM hidden space and appending it as a single conditioning vector. Compare this to feeding a sequence of 16 discrete visual tokens obtained from a VQ-VAE. Which representation gives the LM finer-grained control over *where* information appears in the token stream, and why does that matter for generation?

(d) A student claims: "CLIP is better than VQ-VAE for multimodal language models because CLIP embeddings are more semantic." Give a careful response. Under what downstream goals is this statement reasonable, and under what goals is it incomplete or incorrect?

(e) **Analyse.** In the toy multimodal LM of this assignment, we want the model not only to answer questions about images, but also to generate image token sequences autoregressively. Explain why CLIP/SigLIP alone are insufficient for this purpose. Your answer should address:

- absence of a discrete image vocabulary,
- lack of an invertible or approximately invertible decoder,
- mismatch between global embedding alignment and token-level autoregressive prediction.

(f) Finally, relate this discussion back to catastrophic forgetting. Suppose a frozen CLIP encoder is used only to produce image embeddings for the LM, and only a small projection layer is trained to map those embeddings into the LM hidden space. Would this likely reduce forgetting relative to training a new visual token embedding table jointly inside the LM? Explain the mechanism carefully, and state what capability would be lost in exchange.

3.2 Problem 1: VLM Objectives and Connector Design

The two interface alignment losses used in Part A are:

$$L_{\mathrm{IT}}(\theta) = -\sum_t \log P_\theta(y_t \mid y_{<t},\, x^T,\, V), \tag{1a}$$

$$L_{\mathrm{ITT}}(\theta) = -\sum_t \log P_\theta(y_t \mid y_{<t},\, V). \tag{1b}$$

Problem 1.1 (Interface alignment losses & multi-phase training)

(a) Explain which loss is more appropriate for (i) image captioning, (ii) VQA, and (iii) open-ended image-grounded dialogue. Justify each by identifying the missing conditioning signal and its consequence.

(b) In Phase 1, L_{ITT} is used with the LM backbone frozen. Explain in what sense Phase 1 trains the connector to "speak the LM's language" but does *not* teach the model to

answer questions.

(c) The VQA loss is computed only over answer tokens (all prefix positions receive label -100). Show that computing the loss over visual token positions would be meaningless. What ground-truth label would you assign?

(d) Θ_V (CLIP) is frozen in Phases 1–2. If it were trained from random initialisation during Phase 1, describe two failure modes.

Problem 1.2 (Connector architectures)

MLP projector. $V = \mathrm{GELU}(ZW_1 + b_1)W_2 + b_2$, $W_1 \in \mathbb{R}^{768\times960}$, $W_2 \in \mathbb{R}^{960\times960}$.

(a) Count trainable parameters (include biases). Express as a percentage of SmolLM2's 360 M.

(b) The MLP has no cross-token interaction. Explain why this limits spatial reasoning.

Q-Former. M learnable queries attend to CLIP patches: $V = \mathrm{softmax}(QK_v^\top/\sqrt{d_{\mathrm{LM}}})V_v$.

(c) Show the LM receives exactly M tokens regardless of image resolution.

(d) Explain why very small M risks discarding spatial information. Propose one criterion that forces diverse attention.

(e) Describe the *query collapse* failure mode and one strategy to prevent it.

Gated cross-attention. $\tilde{z}_l = z_l + \tanh(\alpha_l)\,\mathrm{CrossAttn}(z_l, ZW_V)$, with $\alpha_l^{(0)} = 0$.

(f) Count additional parameters per LM layer ($d_{\mathrm{LM}} = 960$, $n_{\mathrm{heads}} = 15$, $d_{\mathrm{head}} = 64$) and for all 24 layers.

(g) This assignment wraps only the **first** LM layer. Explain what this single-layer version can and cannot learn vs. the full Flamingo architecture.

Problem 1.3 (Connector comparison — fill in table)

One precise sentence or formula per cell.

3.3 Problem 2: VQ-VAE Theory

The VQ-VAE training objective is:

$$L_{\mathrm{VQ}} = \underbrace{\|x - D_\psi(z_q)\|_2^2}_{\text{reconstruction}} + \underbrace{\|\mathrm{sg}(z_e) - z_q\|_2^2}_{\text{codebook}} + \beta\,\underbrace{\|z_e - \mathrm{sg}(z_q)\|_2^2}_{\text{commitment}}, \tag{2}$$

where $\mathrm{sg}(\cdot)$ is the stop-gradient operator.

	MLP projector	Q-Former	Gated cross-attn
Visual tokens to LM			
Cross-token interaction			
Trainable params			
Resolution-agnostic?			
LM architecture modified?			
Main failure mode			

Problem 2.1 (Straight-through estimator)

(a) Explain why $\nabla_{z_e}\|z_e - \mathrm{sg}(z_q)\|_2^2$ is well-defined but $\nabla_{z_e}\|x - D_\psi(z_q)\|_2^2$ is not (before any approximation).

(b) The straight-through estimator passes the decoder gradient through the quantiser as if $z_q = z_e$. State the assumption and when it is a good approximation.

(c) Show that $\nabla_\phi L_{\mathrm{VQ}}$ receives contributions from the commitment term and the straight-through reconstruction gradient only. Why does $\|\mathrm{sg}(z_e) - z_q\|_2^2$ contribute nothing to ∇_ϕ?

Problem 2.2 (Codebook perplexity and collapse)

Define $\mathrm{Perp}(\mathcal{C}) = \exp(-\sum_k p_k \log p_k)$, where p_k is the usage fraction of entry k.

(a) Show Perp is maximised at K (uniform) and minimised at 1 (collapse).

(b) Explain mechanistically why very small β promotes collapse and very large β degrades reconstruction. Identify the gradient terms in (2).

(c) EMA updates: $N_k \leftarrow \gamma N_k + (1-\gamma) n_k;\ m_k \leftarrow \gamma m_k + (1-\gamma)\sum_{i:k_i^*=k} z_e^{(i)};\ e_k \leftarrow m_k/N_k$. Explain why EMA improves codebook utilisation early in training.

3.4 Problem 3: Unified Vocabulary and Token Sequences (Part B)

Problem 3.1 (Vocabulary expansion)

After adding $K = 256$ visual tokens and `<image>`/`</image>`, the unified vocabulary has size $V = 49\,410$.

(a) Codebook vectors $e_k \in \mathbb{R}^{64}$ are projected to $\mathbb{R}^{960}$ via a linear projection P. Show

that if P is randomly initialised while the LM's embeddings are well-trained, the VQA gradient initially focuses on P and LoRA. Why is this desirable?

(b) Compare two embedding strategies: (i) *in-place resize*: extend `wte` to size V and make all rows trainable; (ii) *overlay embedding*: keep `wte` frozen, use `nn.Embedding(258, 960)` with a forward hook. Compare along: gradient flow to existing text embeddings, memory cost, implementation complexity.

(c) `<image>` and `</image>` are initialised from the *mean* of existing text embeddings. Explain why this beats random init.

3.5 Problem 4: Catastrophic Forgetting and Mixed Objectives

This problem applies to **both parts**. The mixed objectives are:

$$\text{Part A:} \quad L_{\text{mixed}} = L_{\text{VQA}} + \lambda L_{\text{LM}}, \tag{3a}$$

$$\text{Part B:} \quad L_{\text{mixed}} = L_{\text{VQA}} + \lambda L_{\text{LM}} + \gamma_{\text{img}} L_{\text{IMG}}. \tag{3b}$$

Problem 4.1 (LoRA and forgetting — shared theory)

LoRA: $\Delta W = BA$, $B \in \mathbb{R}^{960\times r}$, $A \in \mathbb{R}^{r\times 960}$.

(a) Show $\text{rank}(\Delta W) \le r$. For $r = 16$, compute the fraction of the 960×960 weight space affected.

(b) For $\lambda = 0$: upper-bound on R as training progresses. For $\lambda \to \infty$: why does VQA accuracy collapse?

(c) Alpaca batches must never contain visual tokens/embeddings. Describe the specific corruption mechanism if visual tokens leak into L_{LM}.

Problem 4.2 (Logit masking and modality routing — Part B)

At inference, the LM produces logits over all $V = 49\,410$ tokens.

(a) For VQA (text output): write the masked logit vector $\tilde{\ell}$ as a function of raw logits ℓ and mask $m \in \{0,1\}^V$. Show that after softmax, masked tokens get zero probability. What numerical value achieves this?

(b) For image generation: propose the complementary mask. Why is it necessary?

(c) Without masking, what fraction of probability mass goes to visual tokens initially?

3.6 Problem 5: Modality Gap and Cross-Architecture Comparison

From lecture notes, InfoNCE decomposes as:

$$L_c(\theta) = \underbrace{-\mathbb{E}[s_{ii}]}_{\text{alignment}} + \underbrace{\mathbb{E}[\log \textstyle\sum_j e^{s_{ij}}]}_{\text{uniformity}}. \tag{4}$$

Problem 5.1 (Gap geometry and mitigation)

(a) Show the uniformity gradient for v_i points away from *all* embeddings but does not specifically push v_i toward text. Why does this promote a persistent modality gap?

(b) Define $\mathrm{MG} = \|\bar{v} - \bar{t}\|_2$ (mean normalised embeddings). Show $0 \leq \mathrm{MG} \leq 2$ and identify the geometric extremes.

(c) Explain the *cone effect*: why does contrastive training push each modality into a narrow cone?

(d) Three mitigation strategies: (I) norm alignment L_{norm}; (II) uniformity regulariser L_{unif}; (III) partial ViT unfreezing. For each: which component it modifies, one failure mode, and one scenario where it outperforms the others.

Coding Section — Part A: Continuous Connector VLMs

4 Coding Tasks — Part A

4.1 Dataset: CIFAR-10 Subset

Dataset	Size (baseline)	Purpose
CIFAR-10 images (train)	10 000 (1 000/class)	Phases 1–3
CIFAR-10 images (test)	2 000 (200/class)	Held-out evaluation
Synthetic captions	10 000 templates	Phase 1 connector init
Synthetic VQA (train)	50 000 (5 templates × 10 000)	Phase 2/3
Synthetic VQA (val)	10 000 (5 templates × 2 000)	Evaluation
Alpaca (replay)	1 000 examples	Language replay

Part A Baseline — Compute-Calibrated for Kaggle T4

- **Vision encoder**: `clip-vit-base-patch32`, frozen, fp32 (≈0.35 GB). Extract 49 patches; discard CLS.
- **LM**: SmolLM2-360M, float16 (≈0.72 GB) + `GradScaler`.
- **Connector (MLP)**: Linear(768,960) – GELU – Linear(960,960). Kaiming init. Verify $r_{\text{norm}} \in [0.5, 2.0]$ before Phase 2.
- **LoRA**: $r = 16$, $\alpha = 32$, targets `q_proj, k_proj, v_proj, o_proj`.
- **Phase 1**: connector only, Adam 3×10^{-4}, batch 32, 1 epoch. *≈5 min.*
- **Phase 2**: connector + LoRA + replay ($\lambda = 0.2$), Adam 5×10^{-4}, grad accum ×4, `OneCycleLR` (10% warmup). *≈15 min.*
- **Phase 3**: connector + LoRA, Adam 2×10^{-4}, 1 epoch. *≈5 min.*
- **Eval**: every 300 steps; 200 VQA pairs, 100 Alpaca for PPL.

Total: ≈25–30 min on T4 (<2.5 GB peak VRAM).

4.2 Task A-C0: Data Pipeline and Model Loading

Task A-C0: Setup

A-C0.1 CIFAR-10 Preprocessing

1. Load `torchvision.datasets.CIFAR10`. Stratified subsets (seed 42): 1 000/class train, 200/class test. Preprocess with `CLIPImageProcessor` (resizes 32 → 224, CLIP normalisation). Cache tensors.
2. Verify CLIP normalisation mean/std differs from ImageNet defaults.

A-C0.2 Caption and VQA Datasets

3. 10 000 captions (six rotating templates per class).
4. Apply all five VQA templates to every image:

Question	Answer	Skill
"What object is shown?"	class name	recognition
"Is there a {class}?"	yes/no	binary
"Vehicle or living thing?"	vehicle / living	abstraction
"Can it fly?"	yes/no	attribute
"Is this an animal?"	yes/no	category

50 000 train / 10 000 val pairs.

A-C0.3 Model Loading

5. Load CLIP (`.requires_grad_(False)`); confirm 50 tokens output.
6. Load SmolLM2 in float16; confirm $d_{\text{LM}} = 960$, vocab 49 152.
7. Load 1 000 Alpaca examples. Compute PPL_0.

Common Pitfalls & Debugging

- **Always use** `CLIPImageProcessor` — raw 32×32 images give silently wrong features.
- **Discard CLS** — use `last_hidden_state[:,1:,:]` (49 spatial patches).
- **Never inject visual tokens into Alpaca batches.**

4.3 Task A-C1: Phase 1 — Connector Initialisation

Task A-C1: Phase 1

Train only Θ_C on caption pairs; CLIP and LM frozen.

1. Sequence: [BOS_emb, $V_{1:49}$, caption_embs] via `inputs_embeds`. Labels: $[-100] \times (1 + 49)$ + caption IDs.
2. Verify $\approx$1.66 M trainable params. Train 1 epoch, batch 32, Adam 3×10^{-4}.
3. Measure r_{norm}. Rescale if outside $[0.3, 3]$.
4. Generate 5 captions from held-out images (greedy, visual tokens only).
5. Save `weights/connector_phaseA1.pt`.

4.4 Task A-C2: Phase 2 — SFT with Language Replay

Task A-C2: Phase 2

Connector + LoRA + Alpaca replay.

1. Apply LoRA ($r = 16$, $\alpha = 32$). Load Phase 1 connector. Trainable $< 1\%$.
2. VQA: [BOS_emb, V, q_embs, a_embs, EOS]. Labels: -100 prefix; answer + EOS receive gradient.
3. $L_{\text{mixed}} = L_{\text{VQA}} + 0.2\, L_{\text{LM}}$. `GradScaler`, grad accum $\times 4$, `OneCycleLR` (10% warmup).
4. Run λ ablation (Table 1).

Table 1: Part A: replay weight λ ablation.

Condition	λ	VQA acc. (%)	R
No replay	0.00	—	—
Weak	0.05	—	—
Baseline	0.20	—	—
Strong	0.50	—	—

4.5 Task A-C3: Phase 3 — VQA Alignment

Task A-C3: Phase 3

Load Phase 2 checkpoint. 1 epoch on 10 000 VQA pairs, no replay ($\lambda = 0$), lr 2×10^{-4}. Compare accuracy vs. Phase 2. Does R increase? Optionally: unfreeze last 4 CLIP ViT blocks (lr 10^{-5}); compare MG. Save `weights/connector_phaseA3.pt`.

4.6 Task A-C4: Evaluation

Task A-C4: VQA Evaluation

1. Exact-match accuracy on 10 000-pair val set: overall, per template, per class.
2. Baselines: (i) text-only (no visual tokens), (ii) majority vote.
3. Plot VQA accuracy and R across Phases 0–3.
4. 6 qualitative examples (2 correct-easy, 2 correct-hard, 2 failures) with top-5 logits.
5. Report peak VRAM, training time, trainable params per phase.

4.7 Task A-C5: Modality Gap Analysis

Task A-C5: Modality Gap

Fix 200 test images and 200 test questions (seed 42). Reuse across all checkpoints.

1. After each phase: compute MG and within/cross-modal cosine similarities.
2. Plot MG across phases. Which phase has the largest effect?
3. UMAP visualisation after each phase.
4. Re-run Phase 2 with $L_{\text{norm}} = (\mathbb{E}\|V_i\|_2 - \mathbb{E}\|T_j\|_2)^2$ (weight 10^{-2}). Does it help?

4.8 Task A-C6: Ablations (choose ≥ 1)

Task A-C6: Ablations

1. Dataset scale: $N \in \{1k, 5k, 10k, 50k\}$.
2. CLS vs. patches vs. mean-pooled.
3. LoRA rank sweep: $r \in \{2, 4, 16, 32\}$.
4. CLIP vs. DINOv2 (`facebook/dinov2-small`, $d_v = 384$, 256 patches).

Coding Section — Part B: Discrete VQ-VAE

5 Coding Tasks — Part B

5.1 Dataset: Synthetic Images

Dataset	Size	Purpose
Synthetic images (train)	4 800 (16×16, 6 classes)	VQ-VAE + image gen
Synthetic images (val)	1 200	VQ-VAE eval
VQA pairs (train)	19 200 (4 templates $\times$ 4 800)	LM fine-tuning
VQA pairs (val)	4 800	LM eval
Image-gen pairs (train)	4 800	Image-generation loss
Alpaca (replay)	1 000	Language replay

Part B Baseline — Compute-Calibrated for Kaggle T4

- **VQ-VAE**: Conv encoder ($3 \to 32 \to 64$, $4\times$ downsampling), $K=256$, $d=64$, $\beta=0.25$, dead-code restart threshold 2. 80 epochs, batch 64, Adam 3×10^{-4}. *≈5 min.*
- **LM**: SmolLM2-360M, float16 + `GradScaler`.
- **LoRA**: $r = 16$, $\alpha = 32$, targets `q/k/v/o_proj`.
- **Embedding**: overlay `nn.Embedding(258, 960)` with hook. LR: overlay 5×10^{-5}; LoRA + projection 5×10^{-4}.
- **Mixed training**: $\lambda = 0.2$, $\gamma_{\text{img}} = 0.5$; sequential fwd/bwd per task; `OneCycleLR` (10% warmup); 3 epochs, batch 16, grad accum $\times4$. *≈90–100 min.*
- **Eval**: every 300 steps; 200 VQA, 100 Alpaca PPL.

Total: ≈100 min on T4 (<3.5 GB peak VRAM).

Practical Nuance (from reference implementation)

Training data bottleneck. Set `steps_per_epoch = max(len(vqa_loader), len(imggen_loader), len(text_loader))` so the epoch length is determined by the *largest* dataset (VQA has 1 200 batches). Using `min` caps at the text loader (62 batches), causing the model to see only ≈26% of VQA data. All loaders use `infinite_loader` (cycle) so smaller datasets wrap around.

Gradient checkpointing vs. KV-cache. Enable `model.gradient_checkpointing_enable()` during training to reduce activation memory. During evaluation, *disable* it before calling `generate_answer` with `use_cache=True` (KV-cache), then re-enable before resuming training. The two are incompatible.

Reference model on CPU. Load the frozen reference model π_{ref} on CPU (saves ≈0.72 GB). Move to GPU only during R evaluation; move back to CPU immediately after.

5.2 Task B-C0: Dataset Generation and Model Loading

Task B-C0: Setup

B-C0.1 Synthetic Image Generation

1. Implement `generate_dataset(n_per_class=1000, seed=42)` for six classes: **spiral**, **triangle**, **circle**, **cross**, **checkerboard**, **gradient**. 80/20 stratified split.

2. Verify: 6×5 grid. Confirm classes are visually distinguishable.

B-C0.2 VQA and Image-Gen Datasets

3. Four VQA templates:

Question	Answer	Skill
"What shape is in this image?"	class name	recognition
"Is there a {class}?"	yes/no	binary
"Geometric or non-geometric?"	geometric/non-geometric	abstraction
"How many axes of symmetry?"	0/1/3/4/infinite	numerical

   19 200 train / 4 800 val pairs.

4. Image-gen: one prompt per image, three cycling templates.

B-C0.3 Model and Alpaca

5. Load SmolLM2 in float16; confirm vocab 49 152, $d_{\text{LM}} = 960$.

6. 1 000 Alpaca examples. Compute PPL_0. Expected: $\approx$8.

Common Pitfalls & Debugging

- **Left-padding.** SmolLM2 is decoder-only; right-padding corrupts batched generation. Set `tokenizer.padding_side = "left"`.

- **Modality-pure batches.** Never shuffle Alpaca into VQA/image-gen batches.

5.3 Task B-C1: VQ-VAE Training and Codebook Analysis

Task B-C1: Train and Analyse VQ-VAE

B-C1.1 Architecture

1. Implement VQ-VAE:

   - **Encoder**: Conv(3→32, k=4, s=2) → Conv(32→64, k=4, s=2) → Conv(64→64, k=3, s=1), each with GroupNorm + ReLU. $z_e \in \mathbb{R}^{4\times4\times64}$.

- **Vector quantiser**: $\mathcal{C} \in \mathbb{R}^{256\times 64}$, init Uniform$(-1/K, 1/K)$. Straight-through estimator. Support both gradient-descent and EMA ($\gamma = 0.99$). Dead-code restart (threshold 2).
- **Decoder**: ConvTranspose2d(64→64) → ConvTranspose2d(64→32) → Conv2d(32→3) + Sigmoid.

Verify ≈187 K params.

B-C1.2 Training

2. Train with loss (2). 80 epochs, batch 64, Adam 3×10^{-4}. Log: recon MSE, codebook loss, perplexity, dead codes.
3. Run four ablations (Table 2).

B-C1.3 Codebook Analysis

4. Quantisation gap $\Delta = L^{\text{post}} - L^{\text{pre}}$.
5. Usage histogram + 256×256 cosine similarity heatmap.
6. Token map visualisation: 6 val images (one per class) with 4×4 index map alongside original and reconstruction.
7. Save best checkpoint as `weights/vqvae_best.pt` (frozen hereafter).

Table 2: VQ-VAE ablation.

Config	K	β	Codebook update	Notes
Baseline	256	0.25	Gradient	—
A	128	0.25	Gradient	Smaller codebook
B	256	1.00	Gradient	Stronger commitment
C	256	0.25	EMA ($\gamma = 0.99$)	EMA vs. gradient

Implementation Guidance

Expected results: recon MSE $\lesssim 0.01$, perplexity ≥ 200, 0 dead codes after 80 epochs with EMA + dead-code restart. $\Delta \approx 1.8$.
Data-driven codebook init (optional): initialise codebook from K-means on a batch of encoder outputs instead of uniform random. This can speed up convergence.
Stop-gradient placement is critical. Codebook loss: sg(z_e) as target. Commitment loss: sg(z_q) as target.
EMA updates are not from `loss.backward()`. Run the EMA formula manually after backward. Do not include codebook parameters in the optimiser when using EMA.

5.4 Task B-C2: Vocabulary Expansion

Task B-C2: Unified Vocabulary

B-C2.1 Vocabulary Expansion

1. Add `<image>`, `</image>`, and $K = 256$ visual tokens. Call `resize_token_embeddings` *before* `get_peft_model`.

   Hint

   Use the **overlay embedding strategy**: keep `wte` frozen, create `nn.Embedding(258, 960)` with a forward hook that swaps in new rows for token IDs $\geq V_{\text{txt}}$. This avoids 47 M extra trainable parameters.

2. Initialise `<image>`/`</image>` rows from the mean of text embeddings. Initialise the $K = 256$ visual token rows to zero (they will be populated by the projector before being made trainable).

3. Verify norm ratio (visual/text) $\in [0.2, 5]$ after projector warm-up. Rescale if needed.

B-C2.2 Two-Phase Projector Warm-Up

4. **Phase 1 – Projector pre-training.** Keep the overlay embedding entirely frozen *except* the two special-token rows (`<image>`, `</image>`). Train only $P : \mathbb{R}^{64} \to \mathbb{R}^{960}$ (Kaiming init) for a few epochs or until $\|P(c_k)\|$ converges relative to text-embedding norms. This forces P to learn a geometrically sound mapping from codebook space to embedding space before any token embedding is exposed to gradient noise.

5. **Phase 2 – Embedding transplant.** After Phase 1, run every codebook vector c_k through the trained P and write the result into the corresponding visual-token row of the overlay embedding:

   $$e_k \leftarrow P(c_k), \quad k = 0, \ldots, K-1.$$

   Mark those K rows as `requires_grad=True` and *discard* P (remove it from the optimiser and from the computational graph). From this point the visual embeddings are free parameters initialised at a meaningful geometry rather than at random or zero.

6. Re-verify the norm ratio after transplant; rescale if outside $[0.2, 5]$.

5.5 Task B-C3: Tokenisation Pipeline

Task B-C3: Tokenisation Pipeline

B-C3 Tokenisation Pipeline

1. `encode_multimodal`: [BOS, `<image>`, $v_{1:16}$, `</image>`, question, answer, EOS]. Labels: -100 prefix; answer + EOS receive gradient.

2. `encode_imagegen`: [BOS, prompt, `<image>`, $v_{1:16}$, `</image>`, EOS]. Labels: -100 prefix

up to `<image>`; visual tokens + `</image>` receive gradient.

3. **Pre-encode all samples** at dataset init; move VQ-VAE to CPU before LM training.
4. Verify: print token-type sequences for 3 images in both modes.

Common Pitfalls & Debugging

- **Index offset**: visual token ID = codebook index + $V_{\text{txt}} + 2$.
- **Resize before LoRA**: `resize_token_embeddings` after `get_peft_model` silently corrupts adapter shapes.
- **Phase boundary**: do *not* let LoRA gradients flow during Phase 1; apply `get_peft_model` only after the embedding transplant (Phase 2) is complete.

5.6 Task B-C4: Mixed-Objective Fine-Tuning

Task B-C4: LoRA Fine-Tuning with Language Replay

B-C4.1 Setup

1. Apply LoRA. Trainable: LoRA + overlay embedding (visual + special rows) + projection P during Phase 1 only. Confirm $< 1\%$ total parameters in Phase 2.
2. LR groups: LoRA at 5×10^{-4}; visual embedding rows at 5×10^{-5}.
3. Reference model π_{ref} frozen on CPU.

B-C4.2 Training Loop

4. **Sequential** forward/backward per task (VQA, then image-gen, then text), each with `scaler.scale(...).backward()`. Weights: $\lambda = 0.2$, $\gamma_{\text{img}} = 0.5$. Single `scaler.step` + `update` after all three.
5. Grad accum $\times4$, `OneCycleLR` (10% warmup). Train 3 epochs.
6. Log every 25 steps. Run λ/γ ablation (Table 3).

B-C4.3 Break-the-Protection

7. LoRA $r = 64$, $\lambda = 0$, $\gamma_{\text{img}} = 0$. Report R and VQA accuracy.

Table 3: Part B mixed training ablation.

Condition	λ	γ_{img}	VQA (%)	R
No replay	0.00	0.00	—	—
Weak	0.05	0.05	—	—
Baseline	0.20	0.50	—	—
Strong	0.50	0.50	—	—

Common Pitfalls & Debugging

- **Sequential vs. joint forward**: sequential uses 1/3 peak activation memory.
- **VQ-VAE on CPU**: pre-encode all samples at dataset init, then free VQ-VAE from GPU.
- **Initial loss sanity**: step-0 L_{VQA} and L_{IMG} should be $\approx \log(49\,410) \approx 10.8$ nats.
- **NaN handling**: early NaN steps are expected with fp16. The `GradScaler` handles them automatically (scale halves). Expect ∼30–40 NaN steps in the first epoch; this is normal.

5.7 Task B-C5: Evaluation

Task B-C5: VQA, Image Generation, and Error Analysis

B-C5.1 VQA Accuracy

1. Evaluate on 500 val pairs (or full 4 800 if time permits). Report: overall, per template, per class. Use KV-cache for fast generation.
2. Compare to text-only and majority-class baselines.
3. Confusion matrix for "What shape?" template.

B-C5.2 Image Generation

4. Generate 12 images (2 per class) autoregressively with image logit mask. Decode via frozen VQ-VAE decoder. Comment on fidelity.

B-C5.3 Logit Masking and Decoding

5. Show before/after logit histograms for one example.
6. Temperature sweep: $T \in \{0.5, 1.0, 1.5\}$.
7. Identify one spatial coherence issue from 1D raster prediction on a 2D grid.

B-C5.4 Qualitative Error Analysis

8. 6 examples: 2 correct VQA, 2 failures, 2 generated images. Show top-5 logits. Link failures to codebook properties.

5.8 Task B-C6: Focused Ablations (choose ≥ 1)

Task B-C6: Ablations

1. Codebook size: $K \in \{64, 128, 256, 512\}$; report Δ and VQA accuracy.
2. Frozen vs. unfrozen VQ-VAE encoder.
3. LoRA rank sweep: $r \in \{4, 8, 32, 64\}$, $\lambda = \gamma = 0$.
4. Loss weight sweep: vary λ and γ_{img} jointly.
5. **No-projector baseline**: skip Phase 1 entirely; initialise all K visual-token rows randomly (Kaiming) and make them trainable from the start. Compare final VQA accuracy and R against the two-phase baseline to quantify the benefit of projector warm-up.
6. **Frozen-embedding baseline**: keep the K visual-token rows *frozen throughout* all of Phase 2 training (special-token rows remain trainable). At the very end, perform a single projector transplant pass ($e_k \leftarrow P(c_k)$) with no subsequent fine-tuning. Compare against the two-phase baseline to isolate how much gain comes from letting transplanted embeddings adapt vs. the projector geometry alone.

Appendix

A. Mathematical Reference

Interface alignment losses. $L_{\text{IT}} = -\sum_t \log P(y_t \mid y_{<t}, x^T, V)$; $L_{\text{ITT}} = -\sum_t \log P(y_t \mid y_{<t}, V)$.

VQ-VAE. $L_{\text{VQ}} = \|x - D_\psi(z_q)\|^2 + \|\text{sg}(z_e) - z_q\|^2 + \beta\|z_e - \text{sg}(z_q)\|^2$.

Codebook perplexity. $\text{Perp}(\mathcal{C}) = \exp(-\sum_k p_k \log p_k)$.

Mixed objectives. Part A: $L = L_{\text{VQA}} + \lambda L_{\text{LM}}$. Part B: $L = L_{\text{VQA}} + \lambda L_{\text{LM}} + \gamma_{\text{img}} L_{\text{IMG}}$.

Forgetting ratio. $R = \text{PPL}(\pi_\theta)/\text{PPL}_0$.

Modality gap. $\text{MG} = \|\bar{v} - \bar{t}\|_2 \in [0, 2]$.

Gated cross-attention. $\tilde{z}_l = z_l + \tanh(\alpha_l)\,\text{CrossAttn}(z_l, ZW_V)$.

Quantisation gap. $\Delta = L^{\text{post}} - L^{\text{pre}} \geq 0$.

B. VRAM Budget (float16)

Component	**A Ph.1**	**A Ph.2/3**	**B LM stage**
CLIP ViT-B/32 (frozen, fp32)	0.35	0.35	—
VQ-VAE (frozen, CPU)	—	—	<0.01
SmolLM2-360M (float16)	0.72	0.72	0.72
LoRA + optimizer (r=16)	—	0.08	0.08
Overlay embed (258 rows)	—	<0.01	<0.01
π_{ref} (CPU)	—	—	0.72 (CPU)
Activations + KV cache	0.5	1.0	1.5
Total GPU (GB)	**≈1.6**	**≈2.2**	**≈2.4**

If OOM: reduce batch to 8 + accum ×8; or use `bitsandbytes` *8-bit quantisation.*

C. Debugging Checklist

Symptom	Cause and fix
VQA loss ≈ 0 at step 0	Labels over visual/question tokens. Only answer positions get label $\neq -100$.
VQ-VAE perplexity < 30	Codebook collapse. Increase β, switch to EMA, or enable dead-code restart.
Phase 1 captions all EOS	Connector norms $10\times$ text norm. Scale by 0.1 or add L_{norm}.
$R > 1.3$ with $\lambda > 0$	Alpaca batches contain visual tokens. Add `is_vision_batch` guard.
MG unchanged across phases	Different 200 examples per measurement. Fix indices at start (seed 42).
CLIP receiving gradients	`.requires_grad_(False)` not called after loading.
Visual IDs in text answers	Logit mask not applied before sampling.
Embedding norms diverge	Overlay LR too high. Use 5×10^{-5}.
`resize_token_embeddings` error	Must resize *before* `get_peft_model`.
NaN loss with float16	Use `torch.amp.autocast` + `GradScaler`.
`no kernel image` on P100	Reinstall PyTorch for CUDA 11.8 (`cu118` wheels); restart runtime.